

AUTOMATED EVAL DATASET GENERATOR

Turning Production LLM Logs Into a Self-Growing Eval Dataset

Project 14 of the AI Engineering Portfolio | Build Guide & Interview Companion
Author: Anas Amiar

"The hardest part of AI evaluation isn't the harness — it's the dataset. Hand-curated golden sets go stale; production traffic is a self-renewing supply of exactly the right test cases."

How to Use This Guide

This guide documents the project exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview. Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

A pipeline that mines production LLM logs into labeled eval test cases: clusters traffic into its natural categories, flags outliers (novel requests, gibberish, prompt-injection attempts — the most valuable candidates), auto-labels each case with quality (from real user signals), difficulty, expected behavior, and assertions, routes low-confidence labels to a human review queue, and dedups against the existing dataset.

Demo over 15 production logs: 4 clusters + 6 outliers found; **11 test cases auto-added, 4 routed to review**; the planted prompt injection labeled adversarial/should_refuse, gibberish labeled should_clarify, off-topic labeled should_refuse — all automatically.

WHY THIS PROJECT LANDS INTERVIEWS

Most teams rely on hand-curated golden sets that go stale within weeks. Solving the data supply problem — turning production traffic into eval data automatically — is a force multiplier AI teams dream about. It's the difference between an eval suite that decays and one that grows while you sleep.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	LogEntry, Cluster, TestCase, PipelineStats
Clustering (mock)	Greedy centroid over BOW	Production swap: embeddings + HDBSCAN
Outlier detection	No-cluster-fit	Injection attempts, gibberish, novel requests
Auto-labeling	Signal-based judge x3 runs	Quality from thumbs-down/retry; agreement = confidence
Confidence routing	≥ 0.8 auto-add	Low-agreement labels go to human review
Dedup	Cosine ≥ 0.85	A thousand refund phrasings become one test case
Scheduler (production)	Nightly cron	The dataset grows while you sleep

The Core Flow

sample logs (signal-boosted) → cluster into categories + surface outliers → label each candidate (quality, difficulty, expected behavior, assertions) with 3-run agreement confidence → dedup against the existing dataset → auto-add or route to review → coverage stats.

Step-by-Step Build Guide

Four components, data-first: the synthetic logs with planted specials came first because the pipeline's value is defined by how it handles exactly those cases.

PHASE 1: Production Logs with Real Signals

Day 1-3

Goal: A log corpus containing everything a mining pipeline must handle.

- 15 support-bot interactions across three natural topics.
- The signals that matter: thumbs-up/down feedback, immediate retries (user rephrased = the answer failed), latency.
- Planted specials: a prompt-injection attempt, gibberish input, an off-topic probe.

Deliverable: data/logs.py — the corpus the pipeline is judged against.

PHASE 2: Clustering + Outlier Detection

Day 3-6

Goal: Find the traffic's natural categories — and the cases that fit none.

- Greedy centroid clustering over bag-of-words vectors (same pipeline shape as embeddings + HDBSCAN).
- Clusters named from their most frequent content word.
- Logs that fit no cluster are OUTLIERS — and all three planted specials landed there, as they should.

Deliverable: cluster_logs() -> (clusters, outliers).

PHASE 3: Auto-Labeling with Confidence Routing

Day 6-9

Goal: Labels good enough to trust — and a queue for the ones that aren't.

- Quality from user signals: thumbs-down + retry → ~1.8 (a negative test case: 'verify the next model fixes this'); thumbs-up → ~4.6.
- Difficulty: simple/moderate/hard/adversarial (injection patterns → adversarial).
- Expected behavior: should_answer / should_refuse / should_clarify.
- 3 noisy labeling runs; agreement → confidence; < 0.8 → review queue.

Deliverable: label_log() -> TestCase with confidence-based status.

Goal: One command that grows the dataset without growing garbage.

- Chain: cluster → label → dedup (cosine $\geq$ 0.85 vs existing cases) → persist.
- Coverage-by-category stats show where the dataset is thin.
- Result: 11 auto-added, 4 to review, 0 garbage labels.

Deliverable: `python3 -m evalgen.pipeline` — the nightly job.

Demo Results

Metric	Result
Logs processed	15 -> 4 clusters + 6 outliers
Auto-added / review	11 / 4 (3-run agreement confidence, threshold 0.8)
Prompt injection	labeled adversarial / should_refuse — automatically
Gibberish input	labeled hard / should_clarify — automatically
Production failures	thumbs-down + retry logs became negative test cases (quality ~1.8)
Coverage stats	per-category counts expose thin areas for targeted collection

Architecture Decisions

Why are outliers the prize instead of noise?

Clustered traffic is what you already know your system does. Outliers are the injection attempts, gibberish, and novel questions no hand-curated set ever contains — exactly what the next model version needs testing against.

Why 3 labeling runs instead of 1?

LLM judges are non-deterministic. Agreement across runs converts that non-determinism into a confidence signal: unanimous auto-adds, disagreement goes to a human. Growth without silently accumulating garbage labels.

Why keep failures instead of excluding them?

A production failure is a perfect test case — inverted. 'The model answered this badly' becomes 'verify the next model answers it well.' Excluding failures throws away the most informative data in the logs.

Deliberately Out of Scope for v1

- Real embeddings + HDBSCAN (the greedy clusterer isolates the same interface).
- Real LLM-as-judge labeling (the agreement mechanism is in place).
- Golden answer / rubric generation per test case.
- The eval runner + regression tracker — that's Project 1, exactly.
- PII redaction on ingestion; the curation dashboard.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Treating outliers as the prize instead of noise. Every clustering pipeline's natural instinct is to keep the clean clusters and discard what doesn't fit — but for eval data that's exactly backwards. The outliers are where the injection attempts, the gibberish, and the novel requests live: the cases no hand-curated golden set ever contains. In my demo all three planted specials landed in the outlier bucket and were labeled correctly without human touch. Second decision worth defending: failures aren't excluded, they're inverted — a thumbs-down answer becomes a negative test case future models must beat.

What I'd Build Next

- Real embeddings + HDBSCAN; real LLM-as-judge labeling.
- Golden answers / rubrics with source verification per case.

- Close the loop: feed output directly into the Model Regression Detection System's golden dataset — production traffic to CI gate.
- The curation dashboard with inter-annotator agreement tracking.

Companion Projects in This Portfolio

The data-supply side of the Model Regression Detection System (Project 1): that project runs a golden dataset against every prompt change; this one keeps the dataset growing and production-representative. The confidence routing is the OCR pipeline's review-queue pattern (Project 4) again.

Repository: github.com/Anas-Amiar/Project-14-eval-dataset-generator